



GuessTheWord

Gruppo 5

Antonio Ametrano – Matr. 0612710145

Hermann Galluccio – Matr. 0612709328

Chiara Iacuzzo – Matr. 0612708797

Francesco Vecchione – Matr. 0612709314

Persistenza dei dati

Il RDBMS utilizzato per il progetto è SQLite e l'interfacciamento con il database avviene attraverso l'API JDBC.

Il package `db`

Contiene il Database Manager e la gerarchia dei Data Access Object (DAO).

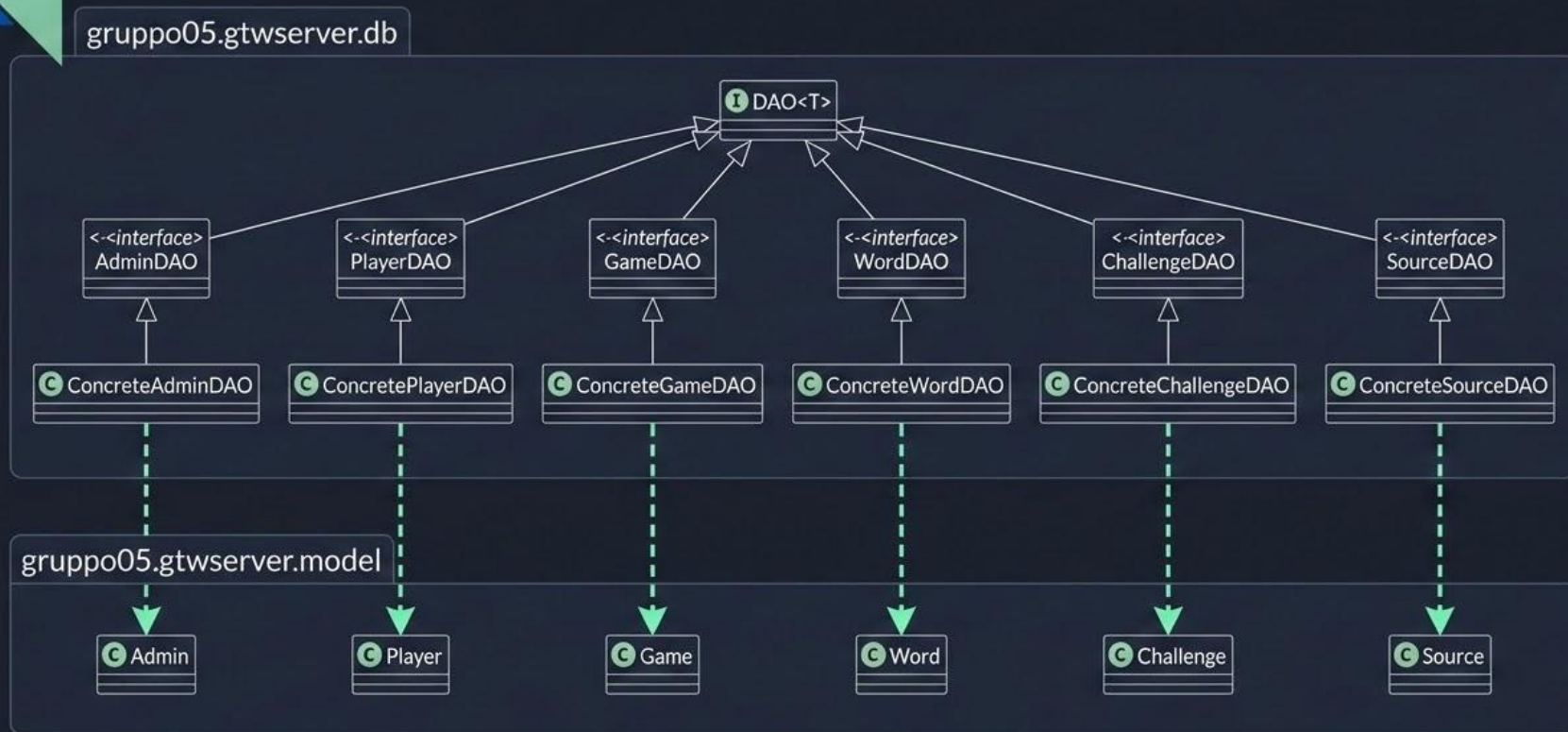


Il package `model`

Contiene gli oggetti immutabili del dominio applicativo.



Gerarchia del DAO





Le regole attive del database

incrementTotalsInPlayer

Permette di incrementare le statistiche all'aggiunta di un record di partita (Game). In particolare, somma i millisecondi giocati al tempo di gioco totale, incrementa il numero di partite effettuate e, nel caso di vittoria, anche il numero di vincite.

deleteOnlyUnreferencedWords

Permette di eliminare tutte le parole (Word) di una certa fonte (Source) su cui non sono state prodotte delle sfide (Challenge). La necessità sorge dal fatto che non si possono perdere le informazioni sulla fonte e le parole usate nella creazione delle sfide nonostante si possa cancellare fisicamente una fonte dal server.

Architettura di Rete **Condivisa**

Analisi tecnica dell'infrastruttura di comunicazione sicura, asincrona e thread-safe.

Panoramica dell'Architettura

- ▶ Progettata per garantire una comunicazione **bidirezionale**, **asincrona** e **thread-safe**.
- ▶ Sfrutta i socket TCP nativi di Java e la serializzazione degli oggetti tramite `ObjectOutputStream` e `ObjectInputStream`.
- ▶ Si **disaccoppia nettamente** dall'interfaccia grafica (UI) mediante l'uso combinato del pattern Builder e di un DTO centralizzato.
- ▶ Previene stati inconsistenti nella vista e standardizza il transito di tutti i messaggi legati al ciclo di vita dell'applicazione.

Aggregazione Flessibile

Risolve il problema degli oggetti parzialmente configurati. Aggrega tutti i possibili campi di rete (credenziali, dati lobby, partita, metriche storico) in un'unica interfaccia pulita. Le variabili interne sono popolate selettivamente in base al `MessageType` ricevuto dal server.

Il ruolo del **CallbackDTO**

Trasporto Sicuro e Immutabile

Data Transfer Object è progettato per fungere da collegamento standardizzato tra il livello di rete e l'interfaccia utente (UI Layout). Implementa il **pattern creazionale Builder** per generare istanze totalmente immutabili di sola lettura (read-only), prevenendo modifiche accidentali.

Aggregazione Flessibile

Risolve il problema degli oggetti parzialmente configurati. Aggrega tutti i possibili campi di rete (credenziali, dati lobby, partita, metriche storico) in un'unica interfaccia pulita. Le variabili interne sono popolate selettivamente in base al `MessageType` ricevuto dal server.

Il Protocollo: **MessageType**



Etichetta Univoca

Un costrutto enumerativo (enum) che definisce formalmente il dizionario dei messaggi ammessi. Assegna un'etichetta immutabile ad ogni pacchetto in transito.



Routing Efficiente

Consente al sistema di routing (Client/Server) di smistare il payload in ricezione in modo ottimizzato. Indirizza istantaneamente il messaggio al controller competente.



Sincronizzazione

MessageType funge da elemento discriminante e punto centrale di sincronizzazione del software. All'arrivo di un pacchetto, il motore di rete esamina questo valore per instradare i dati verso la corretta destinazione.

Categorie di Messaggi

Categoria Logica	Descrizione Operativa
Autenticazione	Coordina i processi di validazione delle credenziali e la creazione di nuovi profili (es. LOGIN_REQUEST, REGISTER_RESPONSE).
Matchmaking	Regola le logiche di abbinamento tra i giocatori. Gestisce l'inserimento degli utenti in code di attesa e notifica in tempo reale le transizioni di stato (es. PLAY_REQUEST, PLAY_RESPONSE).
Ciclo di Partita	Amministra l'evoluzione della sfida, dal trasferimento del payload al risultato finale (es. GAME_START, ANSWER_SUBMISSION).
Connessione	Intercetta disconnessioni volontarie e cadute di rete anomale (es. CLIENT_DISCONNECT, OPPONENT_DISCONNECTED).
Storico	Gestisce la richiesta e l'estrazione dei dati dal DB per fornire lo storico del giocatore (es. HISTORY_REQUEST).
Utility	Transito di messaggi testuali generici, notifiche o comandi da terminale/popup di debug (es. TEXT_MESSAGE).

Connettività Core

NetworkConnection

Il cuore operativo della connettività. NetworkConnection è una classe astratta che modella una connessione multi-canale sicura.

- Nasconde la complessità della concorrenza e dell'I/O di rete.
- Implementa la **tolleranza ai guasti**: ritenta la connessione ogni 5 secondi in caso di fallimento.
- La lista dei thread è thread-safe tramite Collections.synchronizedList.
- **Prevenzione Deadlock**: Impone l'apertura rigorosa dell'ObjectOutputStream *prima* dell'ObjectInputStream.

NetworkConfiguration

Un contenitore di dati immutabile (data class) finalizzato alla centralizzazione dei parametri di rete essenziali all'instaurazione di un canale di comunicazione socket.

Centralizza e rende sicuri i parametri necessari per l'apertura dei socket. Viene tipicamente istanziata durante la fase di bootstrap dell'applicazione dopo la lettura dei file di configurazione.

Il Contratto del Payload: **NetworkMessage**



Classe Base Astratta e Immutabile: Stabilisce il contratto fondamentale per qualsiasi tipo di payload trasportato sulla rete. Ogni pacchetto deve estendere questa classe e implementare Serializable.



Auto-Identificazione: Ogni messaggio viaggiante deve sapersi identificare e deve portare con sé il proprio timestamp di genesi temporale.



Conversione in DTO: Obbliga l'implementazione del metodo astratto toDTO(), permettendo ai ricevitori di convertire agevolmente i pacchetti in oggetti interpretabili dall'interfaccia grafica.



Sottoclassi Specializzate: Incapsula sottoclassi statiche interne (es. LoginRequest, GameResult, ClientDisconnect) per gestire specifiche operazioni di business in modo isolato e tipizzato.

Architettura Client

ClientConnection

- ▶ **ClientConnection:** Implementazione concreta della classe `NetworkConnection` che gestisce un singolo canale bidirezionale (1-a-1) verso il server. Nasconde la complessità multi-canale vincolando le operazioni all'indice fisso 0.
- ▶ **Notifiche Real-time:** Sovrascrive `onReconnected` per avvisare l'utente del ripristino della connessione tramite Alert sul thread JavaFX.

ClientConnectionCreator

- ▶ **ClientConnectionCreator:** Factory e Controller di rete di altissimo livello. Funge da Mediatore tra il TCP grezzo e la UI. Estende la classe `NetworkConnectionCreator`.
- ▶ **Thread Grafico:** Smista i `CallbackDTO` ai controller visivi assicurandosi che l'aggiornamento avvenga *esclusivamente* tramite `Platform.runLater`.

Architettura Server



Gestione Multipla e Sicurezza

La **ServerConnection** governa il ciclo di vita delle connessioni in entrata. costituisce l'implementazione concreta lato server dell'infrastruttura di rete. Estendendo la classe base **NetworkConnection**, specializza il comportamento per accettare e gestire connessioni multiple in ingresso attraverso un **ServerSocket** condiviso.

Il **ServerConnectionCreator** è il Dispatcher universale del backend. Elabora i messaggi dei client, converte in DTO e gestisce le operazioni di autenticazione, la coda di matchmaking, l'avvio partita, la validazione e l'interrogazione dello storico.

ALGORITMO DI CONFRONTO TRA PAROLE

Letter Frequency Similarity

Analisi algoritmica della somiglianza tra parole basata sulla
distribuzione dei caratteri.

Vettorizzazione delle Frequenze

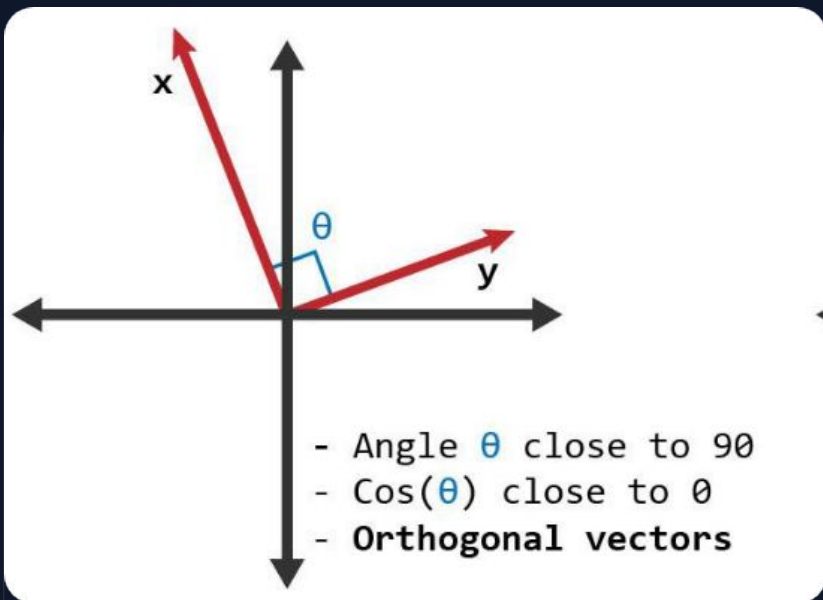
Ogni parola genera un **vettore numerico a 26**

dimensioni, dove ogni indice rappresenta una lettera dell'alfabeto (A-Z).

- ✓ **Normalizzazione:** Conversione automatica in minuscolo (case-insensitive).
- ✓ **Mappatura:** Calcolo dell'indice tramite offset ASCII rispetto al carattere 'a', affinché quest'ultimo si traduca in indice 0.
- ✓ **Filtraggio:** I caratteri speciali e accentati vengono ignorati per ridurre il rumore e semplificare la logica di conversione da carattere a sua rappresentazione numerica.

Codice	Carattere
96	.
97	a
98	b
99	c
100	d
101	e
102	f
103	g
104	h
105	I
106	j
107	k
108	l
109	m
110	n
111	o
112	p
113	q
114	r
115	s
116	t
117	u
118	v
119	w
120	x
121	y
122	z

Core Matematico: Cosine Similarity



La metrica valuta la similarità come l'angolo tra i due vettori di frequenza A e B nello spazio multidimensionale.

$$\text{sim}(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

- ✓ **Prodotto Scalare:** Misura la sovrapposizione delle lettere comuni.
- ✓ **Norme Euclidee:** Rappresentano la "lunghezza" (densità) dei vettori.
- ✓ **Range [0, 1]:** 1.0 indica distribuzioni identiche; 0.0 nessuna lettera in comune.

| Logica di Accettazione



Soglia Hardcoded per Massima Similarità Consentita

Configurata a **0.80**.

Equilibrio ottimale tra flessibilità
morfologica e precisione semantica.



Variazioni Morfologiche

Efficace per riconoscere radici simili e
suffissi diversi.

Esempio: giocatore / giocatori

Implementazione su 3 Principi



Paradigma Funzionale

Utilizzo di *Java Stream API* (`mapToDouble`, `IntStream`) per un codice dichiarativo e pulito, nonché dell'interfaccia funzionale `BiPredicate<String, String>`.



Robustezza Matematica

Gestione preventiva dei casi limite (divisione per zero) su stringhe vuote o non valide.

Se un vettore ha norma zero l'indice non è definito.



Performance

Complessità **$O(n)$** senza mappe onerose, favorendo array semplici per la rappresentazione nello spazio multidimensionale.

Flusso Inizializzazione Partita



Motore Decisionale: **GameSetupController**

Orchestrazione e Conflitti

Agisce come strato di astrazione tra l'infrastruttura di rete e la logica procedurale del gioco. Agisce come il motore decisionale centrale incaricato di orchestrare l'inizializzazione e la configurazione dinamica di ogni sessione di sfida.

Gestisce la **risoluzione dei conflitti** nel caso in cui i partecipanti scelgano difficoltà differenti, applicando un criterio di selezione casuale al 50%. Determina inoltre i parametri temporali operativi (durata del timer) in base alla di.

Generazione del test

La classe delega la complessa generazione testuale a un gestore centralizzato, focalizzandosi sul coordinamento del flusso operativo.

Il costruttore della classe adotta il pattern della Dependency Injection, ricevendo un'istanza centralizzata del SourceManager. Inizializza inoltre i campi interni con valori di fallback predefiniti, al fine di mantenere uno stato coerente e prevenire crash sistemici o stati nulli qualora il database o il generatore non fossero temporaneamente raggiungibili.

Source Manager Component

Architettura, Design Pattern e Ottimizzazione della Business Logic

Asincrona per la Gestione delle Fonti

01. Panoramica Generale

Ecosistema, Responsabilità e Logica di
Business

DESCRIZIONE GENERALE

Modulo responsabile della logica di business associata alle "Sources", operando in modalità asincrona per ottimizzare le prestazioni e non bloccare il thread chiamante.



Persistenza Metadati

Memorizzazione strutturata nel database per garantire integrità e velocità di recupero.



Strategia Ibrida

Metadati su storage persistente; fonti elaborate direttamente in memoria.



Business Logic

Generazione algoritmica avanzata di domande testuali contenenti parole cifrate.

ECOSISTEMA TECNOLOGICO



Java NIO

Accesso efficiente e non bloccante al file system per la lettura delle fonti in tempo reale.



Concurrency Utilities

Utilizzo di ExecutorService per la gestione asincrona dei thread di business.



Stream & Functional

Elaborazione dichiarativa dei dati tramite Stream API e Functional Interfaces.

02. Pattern Architeturali

Struttura Interna e Flessibilità del
Design

PATTERN IMPLEMENTATI



Facade Pattern

Interfaccia pubblica
SourceManager che maschera la
complessità dei sottosistemi
interni.



Builder Pattern

Inizializzazione configurabile
tramite classi Builder per evitare
costruttori complessi.



Callback Pattern

Supporto per operazioni
asincrone non bloccanti tramite
notifiche di esito al chiamante.

PRINCIPI DI DESIGN SOLID

Principio	Applicazione nel Source Manager
SRP (Single Responsibility)	Suddivisione tra SourceAnalyzer, IOManager e QuestionGenerator.
DI & DIP (Dependency)	Iniezione di astrazioni (interfacce) invece di classi concrete.
DRY (Don't Repeat Yourself)	Accentramento della logica I/O nel modulo IOManager dedicato.
OCP (Open/Closed)	Predisposizione all'estensione di algoritmi senza modifica del core.

03. Gestione Eccezioni

Resilienza, Callback Asincrone e Tassonomia degli Errori nel Source
Manager

TASSONOMIA ECCEZIONI TIPIZZATE



Logica & Configurazione

- > **QuestionGenerationException:**
Anomalie impreviste nell'algoritmo di generazione quiz.
- > **PresetNotFoundException:**
Preset di generazione richiesto non presente.



Contenuti (File System)

- > **StopWordsLoadException:**
Fallimento lettura file stop-words.
- > **FrequencyMapNotFoundException:**
Mappa frequenze non reperibile.
- > **SourceNotFoundException:** Fonte inaccessibile.



Storage & Persistenza

- > **StorageException:**
Incapsula errori DAO (connessione, integrità), isolando l'infrastruttura di persistenza.

CANALE DI NOTIFICA ASINCRONO



Natura Asincrona

I fallimenti non utilizzano la propagazione sincrona throws a causa della natura asincrona.



Meccanismo Callback

Notifica al Controller tramite callback **onFailure** basata su interfaccia **Consumer**.



Isolamento & Flusso

Il Controller identifica l'errore senza bloccare il thread principale, garantendo resilienza al pool.

L'architettura garantisce che il fallimento di un singolo thread non comprometta l'intero sistema di esecuzione.

04. Concorrenza e Resilienza

Thread Safety e Gestione delle
Risorse

CONCORRENZA & THREAD SAFETY

Strategia Multithreading



Task Delegation

Orchestrazione dei task tramite ExecutorService dedicato.



Sola Lettura

Accesso thread-safe intrinseco ai contenuti NIO.



Stateless Design

Elaborazione tramite stack dei thread senza stato mutabile.

Persistenza (DAO): Le mutazioni di stato (Add/Delete) sono isolate e delegate a questo layer.

OTTIMIZZAZIONE RISORSE

0

RAM FOOTPRINT
(MB)

Euristiche di Efficienza

- **Skip Heuristic:** Salto posizionale nello stream basato sulla dimensione del file.
- **Short-circuiting:** Consumo pigro (Lazy) tramite Iterator solo per le frasi necessarie.
- **Try-with-resources:** Rilascio immediato dei File Descriptor dal Sistema Operativo.

RESILIENZA E GRACEFUL DEGRADATION

Fallback Semantico

Se i filtri rigidi (stop-words, frequenza) scartano tutti i candidati, il WordExtractor elegge il "meno peggio" basandosi su criteri di fallback configurabili.

Recovery di I/O

In caso di errori nella stima del salto (Skip), l'orchestratore intercetta l'anomalia e riavvia la lettura forzata dall'indice zero per garantire la domanda.